

NomaTune — Spotify Playlist Fix

Ported from ArchiveTune real source · June 2026

Package: com.shahdullah.nomatune

What Was Wrong & What ArchiveTune Has

Bug	Root Cause	Status
Library never shows Spotify tab	ShowSpotifyPlaylistsKey not read by LibraryScreen	✓ FIX 1
LibrarySpotifyPlaylistsScreen missing	Separate screen for Spotify tab never created/wired	✓ FIX 2
SpotifyLibraryViewModel not injected in UI	VM exists but nothing collects playlists StateFlow in library	✓ FIX 3
DataStore cache not restored on init	restoreCachedPlaylists() never called on ViewModel init	✓ FIX 4
Error handler no-op bug	_playlists.value = error.message assigned String to List	✓ FIX 5
isLoading guard skips reload	reloadPlaylists() returns early if isLoading==true	■ KNOWN

FIX 1 — Wire ShowSpotifyPlaylistsKey in LibraryScreen

app/.../ui/screens/library/LibraryScreen.kt

ArchiveTune reads the preference and conditionally adds LibraryFilter.SPOTIFY to the tab list:

```
// Add these imports
import com.shahdullah.nomatune.constants.ShowSpotifyPlaylistsKey
import com.shahdullah.nomatune.utils.rememberPreference
import com.shahdullah.nomatune.constants.LibraryFilter

// Inside LibraryScreen composable:
val (showSpotifyPlaylists) = rememberPreference(ShowSpotifyPlaylistsKey, defaultValue = false)

val libraryFilters = remember(showSpotifyPlaylists) {
    if (showSpotifyPlaylists) {
        listOf(
            LibraryFilter.LIBRARY,
            LibraryFilter.PLAYLISTS,
            LibraryFilter.SPOTIFY,    // <-- conditionally added
            LibraryFilter.SONGS,
            LibraryFilter.ARTISTS,
            LibraryFilter.ALBUMS,
        )
    } else {
        listOf(
            LibraryFilter.LIBRARY,
            LibraryFilter.PLAYLISTS,
            LibraryFilter.SONGS,
            LibraryFilter.ARTISTS,
            LibraryFilter.ALBUMS,
        )
    }
}
```

```
// In the HorizontalPager page routing:
LibraryFilter.SPOTIFY -> {
    LibrarySpotifyPlaylistsScreen(navController = navController)
}
```

Also add *SPOTIFY* case to *headerTitle*, *headerSubtitle*, *tab chip label*, and *tab chip icon* (use *spotify_icon* drawable).

FIX 2 — Create LibrarySpotifyPlaylistsScreen

app/.../ui/screens/library/LibrarySpotifyPlaylistsScreen.kt [NEW FILE]

ArchiveTune has a dedicated screen. Create this file exactly:

```
package com.shahdullah.nomatune.ui.screens.library

import androidx.compose.foundation.layout.*
import androidx.compose.foundation.lazy.*
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Text
import androidx.compose.runtime.*
import androidx.compose.ui.Modifier
import androidx.compose.ui.res.stringResource
import androidx.compose.ui.unit.dp
import androidx.hilt.navigation.compose.hiltViewModel
import androidx.lifecycle.compose.collectAsStateWithLifecycle
import androidx.navigation.NavController
import com.shahdullah.nomatune.LocalPlayerAwareWindowInsets
import com.shahdullah.nomatune.R
import com.shahdullah.nomatune.spotify.SpotifyLibraryViewModel
import com.shahdullah.nomatune.ui.component.ExpressivePullToRefreshBox
import com.shahdullah.nomatune.ui.component.SpotifyLibraryPlaylistListItem

@Composable
fun LibrarySpotifyPlaylistsScreen(
    navController: NavController,
    viewModel: SpotifyLibraryViewModel = hiltViewModel(),
) {
    val playlists by viewModel.playlists.collectAsStateWithLifecycle()
    val isRefreshing by viewModel.isRefreshing.collectAsStateWithLifecycle()
    val playerAwareBottomPadding =
        LocalPlayerAwareWindowInsets.current
            .only(WindowInsetsSides.Bottom)
            .asPaddingValues()
            .calculateBottomPadding() + 12.dp

    ExpressivePullToRefreshBox(
        isRefreshing = isRefreshing,
        onRefresh = viewModel::refreshPlaylists,
        modifier = Modifier.fillMaxSize(),
    ) {
        LazyColumn(
            state = rememberLazyListState(),
            contentPadding = PaddingValues(
                start = 24.dp, end = 24.dp, bottom = playerAwareBottomPadding
            ),
            verticalArrangement = Arrangement.spacedBy(12.dp),
            modifier = Modifier.fillMaxSize(),
        ) {
            if (playlists.isEmpty()) {
                item(key = "spotify_empty") {
                    Text(
                        text = stringResource(R.string.spotify_no_sources),
                        style = MaterialTheme.typography.bodyMedium,
                        color = MaterialTheme.colorScheme.onBackground.copy(alpha = 0.7f),
                        modifier = Modifier.padding(vertical = 16.dp),
                    )
                }
            }
        }
    }
}
```

```

    }
    items(items = playlists, key = { it.id }) { playlist ->
        SpotifyLibraryPlaylistListItem(
            playlist = playlist,
            navController = navController,
        )
    }
}
}
}

```

FIX 3 — Create SpotifyLibraryPlaylistListItem UI component

app/.../ui/component/SpotifyLibraryItems.kt [NEW FILE or add to existing]

ArchiveTune's component navigates to `spotify_playlist/{id}` and shows Spotify icon badge:

```

@Composable
fun SpotifyLibraryPlaylistListItem(
    playlist: SpotifyPlaylist,
    navController: NavController,
    modifier: Modifier = Modifier,
    shape: Shape = RoundedCornerShape(26.dp),
) {
    val libraryPlaylist = remember(playlist) { playlist.toLibraryPlaylist() }
    val openPlaylist = {
        navController.navigate("spotify_playlist/${playlist.id}")
    }
    val trailing: @Composable RowScope.() -> Unit = {
        Icon(
            painter = painterResource(R.drawable.spotify_icon),
            contentDescription = stringResource(R.string.spotify_account),
            tint = MaterialTheme.colorScheme.primary,
            modifier = Modifier.size(20.dp),
        )
    }
    LibraryPlaylistFeatureCard(
        playlist = libraryPlaylist,
        shape = shape,
        trailingContent = trailing,
        modifier = modifier.fillMaxWidth().focusable().clickable(onClick = openPlaylist),
    )
}

// Extension – maps SpotifyPlaylist to your existing Playlist entity for the card
private fun SpotifyPlaylist.toLibraryPlaylist(): Playlist =
    Playlist(
        playlist = PlaylistEntity(
            id = "SPOTIFY_PLAYLIST_${id}",
            name = name,
            thumbnailUrl = images.firstOrNull()?.url,
            remoteSongCount = tracks?.total ?: 0,
            isEditable = false,
        ),
        songCount = tracks?.total ?: 0,
        songThumbnails = images.map { it.url },
    )

```

Note: adapt field names to match your PlaylistEntity. Core pattern is identical to ArchiveTune.

FIX 4 — SpotifyLibraryViewModel — call restoreCachedPlaylists() in init

app/.../spotify/SpotifyLibraryViewModel.kt

ArchiveTune's VM init restores cache so playlists show immediately without a network call:

```

@HiltViewModel
class SpotifyLibraryViewModel @Inject constructor(

```

```

        private val repository: SpotifyLibraryRepository,
    ) : ViewModel() {

        val playlists: StateFlow<List<SpotifyPlaylist>> =
            repository.playlists.stateIn(viewModelScope, SharingStarted.Lazily, emptyList())

        val isRefreshing: StateFlow<Boolean> =
            repository.isRefreshing.stateIn(viewModelScope, SharingStarted.Lazily, false)

        val errorMessage: StateFlow<String?> =
            repository.errorMessage.stateIn(viewModelScope, SharingStarted.Lazily, null)

        init {
            viewModelScope.launch(Dispatchers.IO) {
                repository.restoreCachedPlaylists() // <-- THIS was missing
            }
        }

        fun refreshPlaylists() {
            viewModelScope.launch(Dispatchers.IO) {
                repository.refreshPlaylists()
            }
        }
    }
}

```

FIX 5 — Fix refreshPlaylists() error handler in Repository

app/.../spotify/SpotifyLibraryRepository.kt

ArchiveTune's catch block keeps the cached list on error instead of a no-op assignment:

```

// BEFORE (broken – assigns String to StateFlow<List<...>>)
} catch (error: Throwable) {
    reportException(error)
    _errorMessage.value = error.message
    _playlists.value      // BUG: no-op line does nothing
}

// AFTER (ArchiveTune pattern – returns current cached list)
} catch (error: CancellationException) {
    throw error
} catch (error: Throwable) {
    reportException(error)
    _errorMessage.value = error.message
    _playlists.value    // keep existing cache – DO NOT clear
} finally {
    _isRefreshing.value = false
}

```

Known Issue (ArchiveTune also has this)

reloadPlaylists() in SpotifyAccountViewModel has **if (!_uiState.value.isLoading) return** — this can skip reload if called immediately after connectWithCookies(). ArchiveTune has not fixed this. Workaround: add a separate *isReloading* flag distinct from *isLoading*. Low priority — only affects edge case timing.

Constants / PreferenceKeys Checklist

Ensure these keys exist in your PreferenceKeys.kt:

```

val ShowSpotifyPlaylistsKey = booleanPreferencesKey("showSpotifyPlaylists")
val SpotifyLibraryPlaylistsCacheKey = stringPreferencesKey("spotify_library_playlists_cache")
val SpotifyAccessTokenKey = stringPreferencesKey("spotify_access_token")

```

```
val SpotifyAccessTokenExpiresAtKey = longPreferencesKey("spotify_access_token_expires_at")
val SpotifyAccountNameKey = stringPreferencesKey("spotify_account_name")
val SpotifyAccountAvatarUrlKey = stringPreferencesKey("spotify_account_avatar_url")
val SpotifySpDcKey = stringPreferencesKey("spotify_sp_dc")
val SpotifySpKeyKey = stringPreferencesKey("spotify_sp_key")
```

Also ensure `LibraryFilter.SPOTIFY` exists in your `LibraryFilter` enum/sealed class.

Implementation Order

#	Action	File	Note
1	Fix Repository error handler	<code>SpotifyLibraryRepository.kt</code>	<i>Fix 5 first — stable data layer</i>
2	Fix ViewModel init	<code>SpotifyLibraryViewModel.kt</code>	<i>Fix 4 — cache restore on start</i>
3	Create <code>SpotifyLibraryItems</code> component	<code>ui/component/SpotifyLibraryItems.kt</code>	<i>Fix 3 — needed by new screen</i>
4	Create <code>LibrarySpotifyPlaylistsScreen</code>	<code>ui/screens/library/LibrarySpotifyPlaylistsScreen.kt</code>	<i>Fix 2 — new file</i>
5	Wire <code>LibraryScreen</code>	<code>ui/screens/library/LibraryScreen.kt</code>	<i>Fix 1 — enable the tab</i>
6	Verify constants exist	<code>constants/PreferenceKeys.kt</code> + <code>LibraryFilter</code>	<i>Check checklist above</i>
7	Test	Connect Spotify → Library → Spotify tab	<i>Should show playlists</i>